

The Tool-Agnosticism-Migration Test as Standalone Operational Procedure Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone procedural specification, the operational test that verifies the source paper's tool-agnosticism commitment through actual or simulated migration of a CKS substrate to an alternate vendor.

Abstract

The CKS pattern's tool-agnosticism commitment names three minimal requirements — persistent structured state, human read/write access, and LLM access to substrate content — that any environment hosting a CKS substrate must satisfy, and only those three. Separate derivation notes formalize the commitment and decompose it into operational specializations and composition pairs; none states the operational test by which a deployment can verify the commitment in practice rather than only in claim. This note formalizes that test as a standalone procedural specification. The test is migration-based: a substrate satisfies tool-agnosticism if and only if migrating it to an alternate vendor that meets the three minimal requirements leaves substrate function, authoritative content, governance affordances, provenance fields, and cost-scaling characteristics unchanged. Migration is the most direct empirical exercise of the host-interface specification, because dependencies on host capabilities beyond the three minimal requirements surface at migration time even when they are invisible in abstract specification. The note states what the test verifies, the operational steps that constitute it, the pass and fail conditions, the anti-patterns the test specifically detects, how the test integrates with deployment verification, and its architectural limits. The test admits two modes — actual migration to a real alternate vendor, or simulated migration to a test environment with an alternate vendor — chosen by deployment maturity.

1. Why the migration test needs to be formalized as standalone

The CKS pattern's tool-agnosticism commitment (§7.1 of the source paper) names three minimal requirements for any environment that can host a CKS substrate: persistent structured state, human read/write access, and LLM access to substrate content. A separate derivation note formalizes those three requirements as the load-bearing host-interface specification, and a cluster of operational notes decomposes the commitment into specializations. Composition-pair notes develop the cross-vendor consequences as scalable vendor-independence and as vendor-independent authoritative content. None of these notes states

the *test* by which a deployment can verify the commitment empirically — i.e., that its substrate is in fact a CKS substrate in the architectural sense, rather than a CKS-shaped store with hidden dependencies on a particular vendor's stack.

The gap matters because tool-agnosticism is a commitment whose violation is most often invisible in abstract specification. A deployment that satisfies the three minimal requirements *as stated* may nevertheless rely on host capabilities beyond them — vendor identity-and-access-management as the authority mechanism, vendor configuration systems as the rule store, vendor-specific scaling features for cost predictability, vendor audit-log schemas for provenance. Each such dependency leaves substrate content, governance affordances, or cost behavior bound to the vendor in a way the host-interface specification does not commit to. The dependencies are common because they are convenient; they are also invisible until something actually attempts to remove them.

Migration removes them by force. A substrate migrated to an alternate vendor either continues to function identically — in which case its dependencies on the original were within the three minimal requirements, as the architecture commits — or fails to, in which case the dependencies that broke it are named precisely by what stopped working. Migration is the empirical test that catches what specification cannot. Naming it as standalone procedural specification gives downstream implementers a precise instrument for evaluating the commitment, and gives the architecture's cross-vendor claim a defensible verification method.

This note opens the third cluster of Phase A5 — operational tests for substrate operational properties — following the four-governance-rights tests and the AI-mediation-and-substrate-state tests. Subsequent Phase A5 notes formalize the linear-cost-scaling test, the conflict-coexistence test, and the composition-requirements tests.

2. The architectural commitment under test

The commitment the test verifies is tool-agnosticism — the property that any environment satisfying the three minimal requirements can host a CKS substrate, and that no environment requiring additional capabilities is the host the architecture commits to. The migration test verifies the commitment as a whole rather than any decomposed specialization in isolation.

Under migration, the commitment expands into a set of preservation requirements that any alternate-vendor host must continue to satisfy: substrate function (coordination behavior identical post-migration); rules preserved as substrate-resident content rather than in vendor configuration that does not migrate; provenance fields preserved (writer attribution, timestamp, antecedent reference, rule reference, rationale where applicable, contradiction relationships where applicable); authoritative content preserved across all five source-of-truth categories (decisions, writer attribution, authority references, rationale, unresolved contradictions); cost-scaling characteristics preserved (linear in storage, near-constant in per-query infrastructure cost, no frontier-bending coordination overhead); governance affordances preserved (inspect, modify, override exercisable without scheduling, approval, or runtime intermediation); and mediator role preserved (LLM continues to operate as substrate mediator under human-authored orchestration rules).

Each preservation requirement maps to a separate architectural commitment of the source paper; the migration test verifies that all of them survive together when the host changes.

3. The test procedure as operational steps

The test proceeds in eight steps, applicable in identical form whether the migration is actual or simulated.

(a) Identify substrate components and current vendor. Enumerate the substrate's storage, access mechanisms, rule store, provenance schema, and any auxiliary components. Identify which vendor or vendor-stack hosts each.

(b) Identify alternate vendor with comparable architectural capabilities. Select an alternate whose environment satisfies the three minimal requirements. The alternate need not match the original on operational characteristics; it must satisfy the architectural requirements.

(c) Perform actual or simulated migration of substrate content. In the actual mode, migrate substrate content from the original vendor to the alternate. In the simulated mode, replicate substrate content into an alternate-vendor test environment. Both modes are admissible; deployment-maturity considerations are taken up in §6.

(d) Verify substrate function post-migration. Run the inspect, modify, override, and rule-authoring tests against the alternate-vendor substrate to verify that the four governance rights remain exercisable. Run the substrate-as-source-of-truth test to verify that authoritative content remains authoritative at the alternate vendor.

(e) Verify rules preserved as substrate-resident content. Inspect the alternate-vendor substrate to confirm that orchestration rules are present as substrate content rather than as references to vendor configuration that did not migrate.

(f) Verify provenance fields preserved. Inspect substrate content at the alternate vendor to confirm that all six provenance fields (writer attribution, timestamp, antecedent reference, rule reference, rationale where required, contradiction relationships where required) are present in the same form as at the original vendor.

(g) Verify authoritative content categories preserved. Inspect each of the five source-of-truth categories to confirm preservation across the migration without category loss.

(h) Verify cost-scaling characteristics preserved. Exercise the substrate at representative scale at the alternate vendor; confirm that storage cost grows linearly, per-query infrastructure cost remains near-constant, and no frontier-bending coordination overhead is introduced by the change of vendor.

A vendor-feature inventory accompanies the procedure: any vendor-specific feature the original deployment relied on but that does not migrate is documented. Features that map to architectural commitments are failures; features that do not (operational tooling, monitoring dashboards, vendor-specific developer ergonomics) are recorded as non-architectural.

4. What the test outputs

The test produces a binary architectural verdict, accompanied by a detailed inventory.

Pass. The substrate functions identically at the alternate vendor; rules are preserved as substrate-resident content; all six provenance fields are present in unchanged form; authoritative content is preserved across all five source-of-truth categories; cost-scaling characteristics hold without frontier-bending change; the four

governance rights remain exercisable; the LLM continues to operate as substrate mediator. Any vendor-specific features that did not migrate are documented as non-architectural — they may have been useful at the original vendor and may be useful again, but the substrate's architectural commitments do not depend on them. A pass result is the test verifying that the deployment instantiates tool-agnosticism in fact rather than only in specification.

Fail. Substrate function differs at the alternate vendor; or rules did not migrate as substrate-resident content; or one or more provenance fields are missing or transformed; or one or more source-of-truth categories did not survive; or cost-scaling characteristics changed in ways that suggest dependence on vendor-specific optimizations; or one or more governance rights became unexercisable; or the LLM mediator role broke. A fail result names which preservation requirement failed and which dependency the failure exposes; remediation proceeds by removing the dependency, not by abandoning the migration.

The verdict applies to the deployment's architectural state. A fail does not necessarily mean the deployment is unworkable, but it does mean the deployment does not satisfy the source paper's tool-agnosticism commitment, and any downstream claim that depends on tool-agnosticism — vendor-independence, migration safety, non-specialist governance in commodity tools — is correspondingly weaker.

5. Anti-patterns the test specifically detects

Migration removes invisible dependencies by force, and stating the failure modes precisely is what makes the test diagnostically useful rather than only verdict-producing.

Vendor-specific authority mechanisms. A substrate that depends on the original vendor's identity-and-access-management system, “trusted state” features, or vendor compliance certifications as the architectural source of authority will fail when those mechanisms do not migrate as substrate-resident content. Authority must travel with the substrate; if it lives in the vendor's IAM, the test exposes the binding.

Vendor-specific rule storage. Orchestration rules stored in vendor configuration systems — workflow definition stores, vendor-specific rule engines, infrastructure-as-code manifests — fail to migrate as substrate-resident content. Rules are substrate content per Claim 1; a rule that does not migrate with the substrate was not substrate content in the architectural sense.

Vendor-specific scaling features. Cost-scaling characteristics that depend on proprietary indexing, vendor-managed sharding, or proprietary database optimizations bend at the alternate vendor when those features are absent. The test exposes a cost model inherited from the vendor's infrastructure rather than from the substrate's design.

Substrate substitute. A “substrate” whose function depends on an adjacent component — an agent framework, an orchestration platform, a vendor-managed retrieval system — reveals at migration time that the adjacent component was the actual substrate. The substrate-function preservation requirement fails when the adjacent component does not migrate.

Pure context-window memory as substrate. A “substrate” that is in fact LLM context-window content does not migrate at all; the test exposes the non-migration by finding no addressable structured state to move. The persistent-structured-state requirement should have caught this earlier; the migration test provides the second-line check that catches deployments where context-window dependence had been

masked by an in-session illusion of persistence.

Black-box agent memory as substrate. A “substrate” that is a vendor-specific agent-framework memory system fails the migration test when no equivalent agent framework exists at the alternate vendor or when its memory schema differs. The architectural substrate, in such cases, was the agent framework — not addressable content the human or any other LLM can read.

Vendor-managed governance. Governance affordances that depend on vendor-specific approval workflows, vendor reviewer-roles, or vendor-managed audit dashboards do not migrate as architectural rights. Governance-right exercises (inspect, modify, override) that worked at the original vendor and do not work at the alternate expose vendor-runtime features masquerading as substrate properties.

Vendor-specific provenance schemas. A substrate whose provenance fields are encoded in the original vendor's audit-log schema, with no representation in substrate content itself, fails migration when the alternate vendor's audit-log schema differs. The test exposes provenance that lives in vendor logs rather than as substrate content, which the path-retraceability commitment requires.

The eight anti-patterns are not exhaustive of all possible tool-agnosticism violations, but they are the patterns most commonly produced by deployments that satisfy the three minimal requirements at the surface and bind to a vendor below the surface.

6. How the test integrates with deployment verification

The test admits two modes — actual and simulated — and the choice of mode follows from deployment maturity. Five integration points apply the same procedural specification at different deployment moments rather than as separate tests.

Initial-deployment validation (simulated). Before a substrate is activated for production work, a simulated migration to an alternate vendor verifies that the architecture, as designed, supports the host-interface specification. A failure at this stage is the cheapest moment to catch a tool-agnosticism violation — the deployment can be redesigned before content accumulates.

Pre-migration verification (simulated). Before a planned vendor change, a simulated migration to the prospective alternate vendor verifies that migration will preserve architectural commitments as expected. The simulation surfaces dependencies that have accumulated since initial validation — vendor features adopted incrementally, optimizations that drifted into substrate dependencies, configuration that migrated outside the substrate.

Post-migration verification (actual). After an actual vendor change, the test verifies that the migration in fact preserved what the simulated test predicted. Discrepancies between predicted and actual results are diagnostic: they expose substrate behavior that the simulation did not capture, and the resulting documentation refines subsequent simulations.

Vendor-feature-update verification (simulated). When a vendor releases features the substrate could rely on, a simulated migration verifies that adopting the feature would not introduce a vendor-specific dependency. Features that pass this check can be adopted without compromising tool-agnosticism; features that fail it are documented as non-architectural conveniences.

Composition-partner verification. In multi-substrate compositions, the test runs against each composition partner to verify that migration preserves cross-substrate behavior, not only single-substrate behavior. A composition that passes single-substrate migration tests but fails cross-substrate behavior at the alternate vendor exposes a binding in the composition layer that single-substrate tests do not surface.

7. Limits of the test

The test verifies migration preservation. Stating its limits is what keeps the verdict precise.

It does not verify individual A1.x commitments at the destination in isolation. The destination must satisfy the three minimal requirements as a precondition of the test running at all, but the test does not independently verify that the destination satisfies the four governance rights, the AI-mediator role, the determinism contract, the substrate-cell boundary, or the substrate-as-source-of-truth commitment in their own right. Those are verified by separate Phase A5 tests, run *after* migration to confirm preservation at the destination.

It does not verify cost-scaling preservation in detail. A separate Phase A5 test formalizes the linear-cost-scaling test at the level of operational characterization. The migration test verifies that cost-scaling does not bend across migration; the dedicated cost-scaling test verifies the specific linear-cost claim, including the authoring-cost vs. infrastructure-cost decomposition.

It does not verify operational suitability of the alternate vendor. The alternate may satisfy the three minimal requirements and pass the migration test while remaining unsuitable for production for reasons outside the architecture's scope: performance, support, compliance specifics, ecosystem maturity, organizational policy. The verdict is architectural; operational suitability is a deployment decision the architecture does not adjudicate.

It does not verify migration cost-effectiveness in practice. Migration may be architecturally clean and still costly to perform: data transfer, retraining of operators, integration with existing tooling, parallel-run windows. The verdict is that migration is *possible* without architectural compromise, not that migration is *cheap*.

A test that exceeded these limits would be a different test, and the test's narrow scope is what makes its verdict defensible.

8. One-sentence test

A deployment satisfies the tool-agnosticism commitment if and only if migrating its substrate to an alternate vendor that meets the three minimal requirements leaves substrate function, all six provenance fields, all five source-of-truth categories, the four governance rights, the linear-cost characteristics, and the LLM mediator role unchanged.

9. Why naming the test as standalone matters

The migration test is the most direct empirical exercise of the host-interface specification. Naming it as standalone procedural specification gives downstream implementers an instrument for verifying tool-agnosticism in practice rather than relying on abstract specification alone, and gives the architecture's

cross-vendor claim a defensible verification method that does not depend on particular vendors, particular migration tooling, or particular cloud-portability frameworks. The verification method the test commits to is migration itself; what the test specifies is what to verify, not how to perform a specific vendor's migration.

The test opens Phase A5's third cluster — substrate operational properties — and is followed by the linear-cost-scaling test, the conflict-coexistence test, and the composition-requirements tests. Each subsequent test verifies a different operational property of the substrate; the migration test is distinctive in that it verifies a property — host-interface conformance — most fully exercised when the host actually changes. Specification can describe an interface; only migration confirms the interface holds.

Subsequent work that adopts, extends, or composes the CKS pattern, and that needs to verify tool-agnosticism in deployment, should use the migration test in the form formalized here. Subsequent work that uses different verification methods — abstract conformance reviews, vendor-by-vendor static analysis, feature-by-feature compatibility checks — is verifying a different property, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The Tool-Agnosticism-Migration Test as Standalone Operational Procedure Specification in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.